C++ string 类详解（拼接/查找/替换/截取/长度）

`std::string` 是 C++ 标准库提供的**字符串类**，完全封装了字符数组的底层操作，**不用手动管** `\0`、**不用手动管理内存**，比 C 风格 char 数组更安全、更易用。

一、基础准备

1. 必须包含的头文件

Code block

```
#include <string>   // C++ string 类头文件
#include <iostream>
using namespace std; // 简化写法
```

区分：`<cstring>` 是 C 风格字符串函数（strlen/strcpy），`<string>` 是 C++ string 类。

2. string 初始化方式

Code block

```
string s1;                  // 空字符串
string s2 = "Hello C++";    // 直接赋值
string s3("Hello World");   // 构造函数初始化
string s4(5, 'a');          // 5个a，即 "aaaaa"
```

二、核心常用操作（重点）

1. 长度获取 & 空判断

方法	作用
`s.size()`	获取字符串**有效字符长度**（推荐）

`s.length()`	同 `size()`，完全等价
`s.empty()`	判断是否为空，空返回 `true`

Code block

```
string s = "hello";
cout << s.size() << endl;     // 输出 5
cout << s.length() << endl;   // 输出 5
cout << s.empty() << endl;    // 输出 0 (false)
```

2. 字符串拼接

支持**运算符重载**，比 `strcat` 简单太多，还能拼接 string、char数组、单个字符。

常用方式

Code block

```
string s1 = "Hello";
string s2 = " C++";

// 1. + 拼接（生成新字符串）
string s3 = s1 + s2;          // s3 = "Hello C++"

// 2. += 追加（直接在原串后加）
s1 += " World";               // s1 = "Hello World"
s1 += '!';                    // 追加单个字符

// 3. append() 函数追加
s1.append("!!!");             // 同 +=
```

3. 字符串查找

查找子串/字符的**下标位置**（下标从 0 开始），**没找到返回** `string::npos`（一个特殊常量）。

方法	作用
`s.find(子串)`	从**开头**正向查找第一次出现的位置

`s.rfind(子串)`	从**末尾**反向查找最后一次出现的位置

Code block

```
string s = "abc123abc123";

// 查找子串 "abc"
int pos1 = s.find("abc");   // 找到，返回 0
int pos2 = s.rfind("abc");  // 反向找，返回 6

// 没找到的情况
int pos3 = s.find("xyz");
if (pos3 == string::npos) {
    cout << "未找到" << endl;
}
```

4. 字符串替换

`replace(起始下标， 替换长度， 新子串)`

从指定位置开始，替换指定长度的字符为新字符串。

Code block

```
string s = "I love C";
// 从下标7开始，替换1个字符为"++"
s.replace(7, 1, "++");
// 结果: I love C++
```

5. 字符串截取

`substr(起始下标， 截取长度)`

- 只写起始下标：截取**从该位置到末尾**的所有字符
- 下标从 0 开始

Code block

```
string s = "HelloWorld";

string s1 = s.substr(5, 5);  // 从下标5开始，截5个 → "World"
```

```
string s2 = s.substr(5);     // 从下标5截到末尾 → "World"
```

6. 访问单个字符

和数组一样用 `[]`，也可以用 `at()`（越界会抛异常，更安全）

Code block

```
string s = "hello";
char c = s[1];      // c = 'e'
s[0] = 'H';        // 改为 "Hello"
```

7. 字符串比较

直接用运算符，比 `strcmp` 更直观：

`==`、`!=`、`>`、`<`、`>=`、`<=`

Code block

```
string s1 = "abc";
string s2 = "abd";
cout << (s1 == s2) << endl;  // 0 (false)
cout << (s1 < s2) << endl;   // 1 (true)
```

三、完整可运行示例（整合所有操作）

Code block

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // 1. 初始化
    string s = "Hello C++ Programming";

    // 2. 长度
    cout << "长度: " << s.size() << endl;

```

```
    // 3. 拼接
    s += "!!!";
    cout << "拼接后: " << s << endl;

    // 4. 查找
    int pos = s.find("C++");
    cout << "C++ 起始下标: " << pos << endl;

    // 5. 替换
    s.replace(pos, 3, "Java");
    cout << "替换后: " << s << endl;

    // 6. 截取
    string sub = s.substr(0, 5);
    cout << "截取前5位: " << sub << endl;

    // 7. 空判断
    cout << "是否为空: " << s.empty() << endl;

    return 0;
}
```

运行结果

Code block

```
长度: 21
拼接后: Hello C++ Programming!!!
C++ 起始下标: 6
替换后: Hello Java Programming!!!
截取前5位: Hello
是否为空: 0
```

四、string 对比 char 数组（核心优势）

1. 自动管理内存，**无需手动** `new/delete`
2. 不用关心 `\0` 结束符
3. 拼接/查找/替换语法极简，支持运算符
4. 不会轻易内存溢出，更安全
5. 功能丰富，内置大量实用方法

（注：文档部分内容可能由 AI 生成）